

# 数据结构与算法 B

## common-21 春-期末

### 一、选择题 (30 分, 每小题 2 分)

1. 下列不影响算法时间复杂性的因素有 ( )。

- |           |           |
|-----------|-----------|
| (A) 问题的规模 | (C) 计算结果  |
| (B) 输入值   | (D) 算法的策略 |

2. 链表不具有的特点是 ( )。

- |                  |                   |
|------------------|-------------------|
| (A) 可随机访问任意元素    | (C) 不必事先估计存储空间    |
| (B) 插入和删除不需要移动元素 | (D) 所需空间与线性表长度成正比 |

3. 设有三个元素  $X, Y, Z$  顺序进栈 (进的过程中允许出栈), 下列得不到的出栈排列是 ( )。

- |           |           |
|-----------|-----------|
| (A) $XYZ$ | (C) $ZXY$ |
| (B) $YZX$ | (D) $ZYX$ |

4. 判定一个队列  $Q$  (申请数组空间为  $m$ ) 为空的条件是 ( )。

- (A)  $Q.rear == Q.front$   
 (B)  $Q.front != Q.rear$   
 (C)  $Q.front == (Q.rear+1) \% m$   
 (D)  $Q.front != (Q.rear+1) \% m$

5. 若定义二叉树中根结点的层数为 0, 树的高度等于其结点的最大层数加一。则当某二叉树的前序遍历序列和后序遍历序列正好相反, 则该二叉树一定是 ( )。

- |              |              |
|--------------|--------------|
| (A) 空或只有一个结点 | (C) 任一结点无左孩子 |
| (B) 高度等于其结点数 | (D) 任一结点无右孩子 |

6. 任意一棵二叉树中, 所有叶结点在前序遍历序列、中序遍历序列和后序遍历序列中的相对次序 ( )。

- |           |           |
|-----------|-----------|
| (A) 发生改变  | (C) 不能确定  |
| (B) 不发生改变 | (D) 以上都不对 |

7. 假设线性表中每个元素有两个数据项  $key1$  和  $key2$ , 现对线性表按以下规则进行排序: 先根据  $key1$  的值进行非递减排序; 在  $key1$  值相同的情况下, 再根据  $key2$  的值进行非递减排序。满足这种要求的排序方法是 ( )。

- (A) 先按  $key1$  冒泡排序, 再按  $key2$  直接选择排序  
 (B) 先按  $key2$  冒泡排序, 再按  $key1$  直接选择排序  
 (C) 先按  $key1$  直接选择排序, 再按  $key2$  冒泡排序  
 (D) 先按  $key2$  直接选择排序, 再按  $key1$  冒泡排序

8. 有  $n$  个整数, 找到其中最小整数需要比较次数至少是 ( ) 次。

- (A)  $n$  (C)  $n^2 - 1$   
 (B)  $\log_2 n$  (D)  $n - 1$
9.  $n$  个顶点的无向完全图的边数为 ( )。
- (A)  $n(n - 1)$  (C)  $n(n - 1)/2$   
 (B)  $n(n + 1)$  (D)  $n(n + 1)/2$
10. 设无向图  $G = (V, E)$ ,  $G' = (V', E')$ , 如果  $G'$  是  $G$  的生成树, 则下面说法中错误的是 ( )。
- (A)  $G'$  是  $G$  的子图 (C)  $G'$  是  $G$  的极小连通子图且  $V = V'$   
 (B)  $G'$  是  $G$  的连通分量 (D)  $G'$  是  $G$  的一个无环子图
11. 有一个散列表, 其散列函数为  $h(\text{key}) = \text{key} \bmod 13$ , 使用再散列函数  $h_2(\text{key}) = \text{key} \bmod 3$  解决碰撞。当前散列表状态如下, 其中空白表示该地址尚未存放关键码:

| 地址  | 0 | 1  | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|-----|---|----|---|---|---|---|---|---|---|---|----|----|----|
| 关键码 |   | 38 |   |   |   |   |   |   |   |   |    |    | 25 |

- 从表中检索关键码 38 需进行几次关键码比较 ( )。
- (A) 1  
 (B) 2  
 (C) 3  
 (D) 4
12. 在一棵度为 3 的树中, 度为 3 的结点个数为 2, 度为 2 的结点个数为 1, 则度为 0 的结点个数为 ( )。
- (A) 4 (C) 6  
 (B) 5 (D) 7
13. 由同一组关键字集合构造的各棵二叉排序树 ( )。
- (A) 其形态不一定相同, 但平均查找长度相同 (C) 其形态均相同, 但平均查找长度不一定相同  
 (B) 其形态不一定相同, 平均查找长度也不一定相同 (D) 其形态均相同, 平均查找长度也都相同  
 同
14. 允许表达式内多种括号混合嵌套, 检查表达式中括号是否正确配对的算法, 通常选用 ( )。
- (A) 栈 (C) 队列  
 (B) 线性表 (D) 二叉排序树
15. 下列选项中最适合实现数据的频繁增删及高效查找的组织结构是 ( )。
- (A) 有序表 (C) 二叉排序树  
 (B) 堆排序 (D) 快速排序

## 二、判断题 (10 分, 每小题 1 分; 对 Y, 错 N)

16. 考虑一个长度为  $n$  的顺序表中各个位置插入新元素的概率相同, 则顺序表的插入算法平均时间复杂度为  $O(n)$ 。( )

17. 希尔排序算法的每一趟都要调用一次或多次直接插入排序算法，所以其效率比直接插入排序算法差。( )
18. 直接插入排序、冒泡排序、Shell 排序都是在数据正序的情况下比数据在逆序的情况下要快。( )
19. 由于碰撞的发生，基于散列表的检索仍然需要进行关键码对比，并且关键码的比较次数仅取决于选择的散列函数与处理碰撞的方法两个因素。( )
20. 若有一个叶子结点是二叉树中某个子树的前序遍历结果序列的最后一个结点，则它一定是该子树的中序遍历结果序列的最后一个结点。( )
21. 若某非空二叉树的前序遍历序列和后序遍历序列正好相同，则该二叉树只有一个根结点。( )
22. 有  $n$  个结点的二叉排序树有多种；若按最坏情况下的查找比较次数衡量，树高最小的二叉排序树搜索效率最好。( )
23. 强连通分量是有向图中的最小强连通子图。( )
24. 用邻接矩阵法存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中结点个数有关，而与图的边数无关。( )
25. 若定义一个有向图的根是指可以从这个结点到达图中任意其他结点，则可知一个有向图中至少有一个根。( )

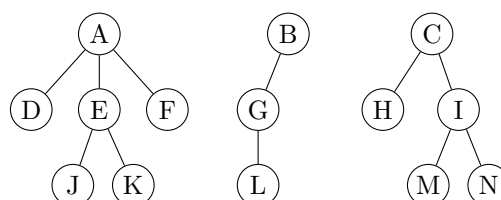
### 三、填空题 (20 分，每题 2 分)

26. 线性表的顺序存储与链式存储是两种常见存储形式；当表元素有序排序进行二分检索时，应采用 \_\_\_\_\_ 存储形式。
27. 设环形队列的容量为 20(单元编号 0 到 19)，经过一系列的入队和出队运算后，队头变量  $front=18$ ，队尾变量  $rear=11$ ，则环形队列中有 \_\_\_\_\_ 个元素。
28. 一棵含有 101 个结点的二叉树中有 36 个叶子结点，度为 2 的结点个数是 \_\_\_\_\_，度为 1 的结点个数是 \_\_\_\_\_。
29. 已知二叉树的前序遍历结果为  $ADC$ ，这棵二叉树的树型有 \_\_\_\_\_ 种可能。
30. 已知二叉树的中序序列为  $DGBAECF$ ，后序序列为  $GDBEFCA$ ，该二叉树的先序序列是 \_\_\_\_\_。
31. 对于具有 57 个结点的完全二叉树，按层次自顶向下、同一层自左向右、从 0 开始编号。编号为 18 的结点的父结点编号为 \_\_\_\_\_，编号为 19 的结点的右子女结点编号为 \_\_\_\_\_。
32. 有  $n$  个数据对象的二路归并排序中，每趟归并的时间复杂度为 \_\_\_\_\_。
33. 对一组记录进行降序排序，其关键码为 (46, 70, 56, 38, 40, 80, 60, 22)，采用初始步长为 4 的希尔排序，第一趟扫描的结果是 \_\_\_\_\_，而采用归并排序第一轮归并的结果是 \_\_\_\_\_。
34. 如果只想得到 1000 个元素的序列中最小的前 5 个元素，在冒泡排序、快速排序、堆排序和归并排序中，哪种算法最快？ \_\_\_\_\_
35. 如果一个图结点多而边少 (稀疏图)，适宜采用 \_\_\_\_\_ 方式进行存储。

### 四、简答题 (24 分，每小题 6 分)

#### 36. (森林周游) (6 分)

树周游算法可以很好地应用到森林的周游上。查看下列森林结构，请给出其深度优先周游序列和广度优先周游序列。

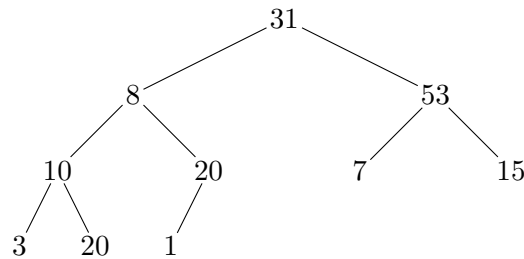


**37. (Huffman 树) (6 分)**

设给定五个字符，其相应的权值分别为  $\{4, 8, 6, 9, 18\}$ ，试画出相应的哈夫曼树，并计算它的带权外部路径长度 WPL。

**38. (堆操作) (6 分)**

下图是一棵完全二叉树：



- 1) 请根据初始建堆算法对该完全二叉树建堆，请画出构建的小根堆；(2 分)
- 2) 基于 (1) 中得到的堆，删除其中的最小元素，请用图给出堆的调整过程；(2 分)
- 3) 基于 (1) 中得到的堆，向其中插入元素 2，请给出堆的调整过程。(2 分)

**39. (Prim 算法求最小生成树) (6 分)**

已知图  $G$  的顶点集合  $V = \{V_0, V_1, V_2, V_3, V_4\}$ ，邻接矩阵如下 ( $\infty$  表示无边)：

$$\begin{pmatrix} 0 & 7 & \infty & 4 & 2 \\ 7 & 0 & 9 & 1 & 5 \\ \infty & 9 & 0 & 3 & \infty \\ 4 & 1 & 3 & 0 & 10 \\ 2 & 5 & \infty & 10 & 0 \end{pmatrix}$$

- 1) 根据 Prim 算法，求图  $G$  从顶点  $V_0$  出发的最小生成树；(2 分)
- 2) 给出每一步执行后的 mst 数组。(4 分)

**五、算法题 (16 分，每小题 8 分)**

**40. (根据前序求后序)** 下列 Python 函数的功能是：根据一个满二叉树的前序遍历序列，求其后序遍历序列。参数  $s$  为当前子树在前序序列中的起点， $length$  为当前子树结点数。请补全空白 (假设序列长度不超过 32)。

```

def preorder_to_postorder(seq, s, length):
    if length == 1:
        return _____ # (1)

    sub_len = _____ # (2)
    s1 = s + 1
    s2 = _____ # (3)
    left = preorder_to_postorder(seq, s1, sub_len)
    right = _____ # (4)
    return _____ # (5)
    
```

**41. (深度优先周游)** 阅读下列 Python 程序，完成图的深度优先周游算法。图采用邻接表表示：  
`graph["vertices"]` 存放顶点标记，`graph["adj"][i]` 存放顶点  $i$  的所有邻接点编号。

```

def dfs_in_list(graph, visited, i):
    print(f"node: {graph['vertices'][i]}")
    visited[i] = True
    
```

```
    for j in _____:                # (1)
        if _____:                # (2)
            _____                # (3)

def traverse_dfs(graph):
    n = len(graph["vertices"])
    visited = [False] * n
    for i in range(n):
        if _____:                # (4)
            dfs_in_list(graph, visited, i)
```

## 数据结构与算法 B

## common-22 春-期末

## 一、选择题 (30 分, 每小题 2 分)

1. 双向链表中的每个结点有两个引用域 `prev` 和 `next`, 分别引用当前结点的前驱与后继。设 `p` 引用链表中的一个结点, `q` 引用一待插入结点, 现要求在 `p` 前插入 `q`, 正确的插入操作为 ( )。
- (A) `p.prev=q; q.next=p; p.prev.next=q; q.prev=p.prev;`  
(B) `q.prev=p.prev; p.prev.next=q; q.next=p; p.prev=q.next;`  
(C) `q.next=p; p.next=q; p.prev.next=q; q.next=p;`  
(D) `p.prev.next=q; q.next=p; q.prev=p.prev; p.prev=q;`
2. 给定一个  $N$  个相异元素构成的有序数列, 设计递归算法实现二分查找。考察递归过程中栈的使用情况, 该递归调用栈的最小容量应为 ( )。
- (A)  $N$  (C)  $\lfloor \log_2 N \rfloor$   
(B)  $N/2$  (D)  $\lceil \log_2(N+1) \rceil$
3. 数据结构有三个基本要素: 逻辑结构、存储结构以及基于结构定义的行为 (运算)。下列概念中, ( ) 属于存储结构。
- (A) 线性表 (C) 字符串  
(B) 链表 (D) 二叉树
4. 采用数组  $Q[0..m-1]$  实现循环队列, 变量 `rear` 表示队尾元素的实际位置, 添加结点时按 `rear = (rear + 1) % m` 移动, 变量 `length` 表示当前队列中的元素个数, 则队首元素的实际位置是 ( )。
- (A) `rear - length`  
(B) `(1 + rear + m - length) % m`  
(C) `(rear - length + m) % m`  
(D) `m - length`
5. 给定一棵二叉树, 若前序遍历序列与中序遍历序列相同, 则该二叉树是 ( )。
- (A) 根结点无左子树的二叉树 的二叉树  
(B) 根结点无右子树的二叉树 (D) 只有根结点的二叉树或非叶子结点只有右子树的二叉树  
(C) 只有根结点的二叉树或非叶子结点只有左子树 的二叉树
6. 用 Huffman 算法构造最优二叉编码树, 待编码字符权值为  $\{3, 4, 5, 6, 8, 9, 11, 12\}$ , 该树的带权外部路径长度为 ( )。
- (A) 58 (C) 72  
(B) 169 (D) 182
7. 若需要对存储开销约 1GB 的数据排序, 而当前可用 RAM 只有 100MB, 最适合的排序算法是 ( )。
- (A) 堆排序 (C) 快速排序  
(B) 归并排序 (D) 插入排序

8. 一个无向图  $G$  含有 18 条边，其中度数为 4 的顶点有 3 个，度数为 3 的顶点有 4 个，其他顶点的度数均小于 3，则  $G$  的顶点数至少为 ( )。

- (A) 10
- (B) 11
- (C) 13
- (D) 15

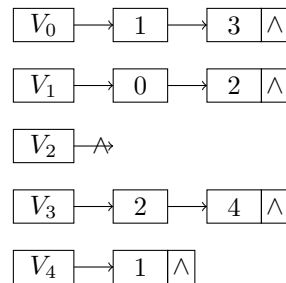
9. 无向图  $G$  从  $V_0$  出发作深度优先遍历，访问的树边集合为

$$\{(V_0, V_1), (V_0, V_4), (V_1, V_2), (V_1, V_3), (V_4, V_5), (V_5, V_6)\}.$$

则下列哪条边不可能出现在  $G$  中? ( )

- (A)  $(V_0, V_2)$
- (B)  $(V_4, V_6)$
- (C)  $(V_4, V_3)$
- (D)  $(V_0, V_6)$

10. 已知有向图  $G$  的邻接边表如下图所示，



从  $V_0$  出发进行深度优先周游，得到的 DFS 结点序列为 ( )。

- (A)  $V_0, V_1, V_4, V_3, V_2$
- (B)  $V_0, V_1, V_2, V_3, V_4$
- (C)  $V_0, V_1, V_3, V_2, V_4$
- (D)  $V_0, V_2, V_1, V_3, V_4$

11. 若按排序的稳定性对排序算法分类，下列算法中不稳定的是 ( )。

- (A) 冒泡排序
- (B) 归并排序
- (C) 直接插入排序
- (D) 希尔排序

12. 下列哪组排序算法的最好情况下时间复杂度的大  $O$  表示相同? ( )

- (A) 快速排序和堆排序
- (B) 归并排序和插入排序
- (C) 快速排序和选择排序
- (D) 堆排序和冒泡排序

13. 设结点  $X$  和  $Y$  是二叉树中任意两个结点。若在该树的先根周游序列中  $X$  出现在  $Y$  之前，而在其后根周游序列中  $X$  出现在  $Y$  之后，则  $X$  与  $Y$  的关系是 ( )。

- (A)  $X$  是  $Y$  的左兄弟
- (B)  $X$  是  $Y$  的右兄弟
- (C)  $X$  是  $Y$  的祖先
- (D)  $X$  是  $Y$  的后裔

14. 森林  $F$  中每个结点的子结点数均不超过 2。若森林  $F$  中叶子结点总数为  $L$ ，度数为 2 的结点总数为  $N$ ，则森林  $F$  中树的个数为 ( )。

- (A)  $L - N - 1$
- (B) 无法确定
- (C)  $L - N$
- (D)  $N - L$

15. 回溯法是一类广泛使用的算法，下列叙述中不正确的是 ( )。

- (A) 回溯法可以系统地搜索一个问题的所有解或者任意解
- (B) 回溯法是一种既具系统性又具跳跃性的搜索算法
- (C) 回溯算法需要借助队列数据结构来保存从根结点到当前扩展结点的路径
- (D) 回溯法在生成解空间的任一结点时，先判断当前结点是否可能包含问题的有效解；若肯定不包含，则跳过对该结点为根的子树的搜索，逐层向祖先结点回溯

## 二、判断题 (10 分，每小题 1 分；对填 Y，错填 N)

16. 对任意一个连通的、无环的无向图，从图中移除任何一条边得到的图均不连通。( )
17. 给定二叉树，前序周游序列为 HGEDBFCA、中序周游序列为 EGBDHFAC 时，其后序周游序列必为 EBDGACFH。( )
18. 对结点值在 1 到 1000 之间的二叉搜索树查找 363，以下三个序列皆可能为查找路径：  
 A. 2, 252, 401, 398, 330, 344, 397, 363;  
 B. 925, 202, 911, 240, 912, 245, 363;  
 C. 935, 278, 347, 621, 299, 392, 358, 363.  
 ( )
19. 构建一个含  $N$  个结点的 (二叉) 最小值堆，建堆时间复杂度的大  $O$  表示为  $O(N \log N)$ 。( )
20. 队列是动态集合，其定义的出队列操作所移除的元素总是在集合中存在时间最长的元素。( )
21. 任一有向图的拓扑序列既可以通过深度优先搜索求解，也可以通过宽度优先搜索求解。( )
22. 对任一连通无向图  $G$ ，其中  $E$  是权值最小的边，那么  $E$  必然属于任何一个最小生成树。( )
23. 对一个包含负权值边的图，Dijkstra 算法总能够给出最短路径问题的正确答案。( )
24. 分治算法通常将原问题分解为几个规模较小但类似于原问题的子问题，递归求解这些子问题，然后再合并子问题的解来建立原问题的解。( )
25. 考察某个具体问题是否适合应用动态规划算法，必须判定它是否具有最优子结构性质。( )

## 三、填空题 (20 分，每题 2 分)

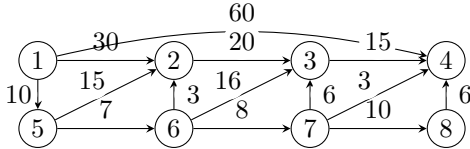
26. 目标串长为  $n$ 、模式串长为  $m$ ，朴素模式匹配算法中字符最多比较次数为 \_\_\_\_\_。
27. 一棵含  $n$  个结点的树中，只有度为  $k$  的分支结点和度为 0 的终端结点，则终端结点数为 \_\_\_\_\_。
28. 对关键码序列 {46, 70, 56, 38, 40, 80} 作快速排序，以第一个记录为基准的一次划分结果为 \_\_\_\_\_。
29. 长度为 3 的顺序表中，查找第一个、第二个、第三个元素的概率分别为  $1/2, 1/4, 1/8$ 。若剩余概率理解为查找失败且失败需比较 3 次，则平均比较次数为 \_\_\_\_\_。
30. 关键码为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，用链地址法和  $H(\text{key}) = \text{key} \bmod 13$  构造散列表，则地址为 1 的链中有 \_\_\_\_\_ 个记录。
31. 删除长度为  $n$  的顺序表第  $i$  个数据元素需要移动表中的 \_\_\_\_\_ 个数据元素。
32. 小根堆为 [8, 15, 10, 21, 34, 16, 12]，删除关键字 8 并重新建堆时，关键字比较次数为 \_\_\_\_\_。



33. 在广度优先遍历、拓扑排序、求最短路径三种算法中，可判断有向图是否有环的是 \_\_\_\_\_。
34.  $n$  ( $n \geq 2$ ) 个顶点的有向强连通图最少有 \_\_\_\_\_ 条边。
35. 栈  $S_1$  中操作数自栈底到栈顶为 5, 8, 3, 2，栈  $S_2$  中运算符自栈底到栈顶为 \*, -, /，按题给函数调用三次后，若按整数除法计算， $S_1$  栈顶为 \_\_\_\_\_。

四、简答题 (3 题，共 20 分)

36. (Dijkstra 算法) 试用 Dijkstra 算法求题图中顶点 1 到其余各顶点的最短路径，写出算法执行过程中各步状态。



| 所选顶点 | $U$ (已确定最短路径的顶点集合) | $T$ (未确定最短路径的顶点集合)    | $d_2$ | $d_3$    | $d_4$ | $d_5$ | $d_6$    | $d_7$    | $d_8$    |
|------|--------------------|-----------------------|-------|----------|-------|-------|----------|----------|----------|
| 初始   | {1}                | {2, 3, 4, 5, 6, 7, 8} | 30    | $\infty$ | 60    | 10    | $\infty$ | $\infty$ | $\infty$ |

37. (二叉搜索树) 给定关键码集合 {18, 73, 5, 10, 68, 99, 27}。
- (1) 画出按记录顺序构造形成的二叉排序 (搜索) 树；
  - (2) 画出删除关键码 73 后的二叉排序树；
  - (3) 画出对集合  $C$  (未删除 73 前) 查询效率最高的最优二叉排序树，其中每个关键码被查询概率相等。
38. (奇偶交换排序) 奇偶交换排序如下：第一趟对所有奇数  $i$  比较  $a_i$  与  $a_{i+1}$ ，必要时交换；第二趟对所有偶数  $i$  比较  $a_i$  与  $a_{i+1}$ ，必要时交换；如此交替，直到整个序列有序。对序列 {18, 73, 5, 10, 68, 99, 27, 10} 写出前 4 趟结果，说明算法是否稳定，并给出正序、逆序输入时的比较次数和交换次数。

五、算法填空 (2 题，共 20 分)

39. (单链表逆置) 填空完成程序：读入整数序列，用单链表存储，然后将该单链表原地逆置后输出。下面程序使用普通头引用 head，链表结点定义为 Node(data)。

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def print_list(head):
    p = head
    while p is not None:
        print(p.data, end=" ")
        p = p.next
    print()

def reverse_list(head):
    if head is not None:
        p = head.next
```

```

        head.next = _____ # (1)
        while p is not None:
            q = p
            p = _____ # (2)
            q.next = _____ # (3)
            _____ # (4)
        return head

n = int(input())
a = list(map(int, input().split()))
head = Node(a[0])
p = head
for i in range(1, n):
    p.next = _____ # (5)
    p = _____ # (6)

head = reverse_list(head)
print_list(head)

```

40. (由扩充二叉树先序序列构造二叉树) 输入一棵二叉树的扩充二叉树先序序列，其中 @ 表示空结点。递归构造该二叉树，并输出其中序遍历序列。

例如输入 ABD@@G@@CE@@F@@ 时，对应二叉树的中序遍历为 DGBAECF。

```

class BinaryTreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

s = input().strip()
pos = 0

def build_tree():
    global pos
    ch = s[pos]
    pos += 1
    if ch == "@":
        return _____ # (1)

    root = _____ # (2)
    root.left = _____ # (3)
    root.right = _____ # (4)
    return root

def inorder(root):
    if root is None:
        return

    _____ # (5)
    print(root.data, end=" ")
    _____ # (6)

tree = build_tree()
inorder(tree)

```

# 数据结构与算法 B

## common-23 春-期末

### 一、选择题 (30 分, 每小题 2 分)

1. 下列叙述中正确的是 ( )。

- (A) 散列是一种基于索引的逻辑结构
- (B) 基于顺序表实现的逻辑结构属于线性结构
- (C) 数据结构设计影响算法效率, 逻辑结构起到了决定作用
- (D) 一个逻辑结构可以有多种类型的存储结构, 且不同类型的存储结构会直接影响到数据处理效率

2. 在广度优先搜索算法中, 一般使用什么辅助数据结构? ( )

- (A) 队列
- (B) 栈
- (C) 树
- (D) 散列

3. 若某线性表实现采取链式存储, 那么该线性表中结点的存储地址 ( )。

- (A) 一定不连续
- (B) 既可连续亦可不连续
- (C) 一定连续
- (D) 与头结点存储地址保持连续

4. 若某线性表的常用操作是在表尾插入或删除元素, 则时间开销最小的存储方式是 ( )。

- (A) 单链表
- (B) 仅有头引用的单循环链表
- (C) 顺序表
- (D) 仅有尾引用的单循环链表

5. 给定后缀表达式 (逆波兰式)  $ab+-c*d-$ , 其对应的中缀表达式是 ( )。

- (A)  $a-b-c*d$
- (B)  $-(a+b)*c-d$
- (C)  $a+b*c-d$
- (D)  $(a+b)*(-c-d)$

6. 今有一空栈  $S$ , 基于待进栈的数据元素序列  $a, b, c, d, e, f$ , 依次进行进栈、进栈、出栈、进栈、进栈、出栈的一系列操作, 上述操作完成以后, 栈  $S$  的栈顶元素为 ( )。

- (A)  $f$
- (B)  $c$
- (C)  $a$
- (D)  $b$

7. 对于带头结点的单链表, 其表头为  $head$ , 则判定空表的条件是 ( )。

- (A)  $head$  is None
- (B)  $head$  is not None
- (C)  $head.next$  is  $head$
- (D)  $head.next$  is None

8. 以下典型排序算法中, 具有稳定排序特性的是 ( )。

- (A) 冒泡排序 (Bubble Sort)
- (B) 直接选择排序 (Selection Sort)
- (C) 快速排序 (Quick Sort)

- (D) 希尔排序 (Shell Sort)
9. 以下关键字序列中，可以有效构成一个大根堆 (即最大值堆，最大值在堆顶) 的序列是 ( )。
- (A) 5, 8, 1, 3, 9, 6, 2, 7  
(B) 9, 8, 1, 7, 5, 6, 2, 3  
(C) 9, 8, 6, 3, 5, 1, 2, 7  
(D) 9, 8, 6, 7, 5, 1, 2, 3
10. 以下典型排序算法中，内存开销最大的是 ( )。
- (A) 冒泡排序 (C) 归并排序  
(B) 快速排序 (D) 堆排序
11. 排序算法往往依赖于对待排序元素序列的多趟比较/移动操作 (即执行多轮循环)，第一趟结束后，任一元素都无法确定其最终排序位置的算法是 ( )。
- (A) 选择排序 (C) 冒泡排序  
(B) 快速排序 (D) 插入排序
12. 考察以下基于单链表的操作，相较于顺序表实现，带来更高时间复杂度的操作是 ( )。
- (A) 合并两个有序线性表，并保持合成后的线性表依然有序  
(B) 交换第一个元素与第二个元素的值  
(C) 查找某一元素值是否在线性表中出现  
(D) 输出第  $i$  个 ( $0 \leq i < n$ ,  $n$  为元素个数) 元素
13. 已知一个整型数组中的元素值依次为 {19, 20, 50, 61, 73, 85, 11, 39}，采用某种排序算法，在多趟比较/移动操作后，依次得到如下中间结果：
- (1) 19 20 11 39 73 85 50 61,  
(2) 11 20 19 39 50 61 73 85,  
(3) 11 19 20 39 50 61 73 85.
- 请问，上述过程使用的排序算法是 ( ) 算法。
- (A) 冒泡排序 (B) 插入排序 (C) 希尔排序 (D) 归并排序
14. 今有一非连通无向图，共有 36 条边，该图至少有 ( ) 个顶点。
- (A) 8 (B) 9 (C) 10 (D) 11
15. 令  $G = (V, E)$  是一个无向图，若  $G$  中任何两个顶点之间均存在唯一的简单路径相连，则下面说法中错误的是 ( )。
- (A) 图  $G$  中添加任何一条边，不一定造成图包含一个环  
(B) 图  $G$  中移除任意一条边得到的图均不连通  
(C) 图  $G$  的逻辑结构实际上退化为树结构  
(D) 图  $G$  中边的数目一定等于顶点数目减 1

## 二、判断题 (10 分, 每小题 1 分; 对填写 Y, 错填写 N)

16. 按照前序、中序、后序方式周游一棵二叉树, 分别得到不同的结点周游序列, 然而三种不同的周游序列中, 叶子结点都将以相同的顺序出现。( )
17. 构建一个含  $N$  个结点的 (二叉) 最小值堆, 时间效率最优情况下的时间复杂度大  $O$  表示为  $O(N \log N)$ 。( )
18. 对任意一个连通的无向图, 如果存在一个环, 且这个环中的某一条边的权值不小于该环中任意一个其它的边的权值, 那么这条边一定不会是该无向图的最小生成树中的边。( )
19. 通过树的周游可以求得树的高度, 若采取深度优先遍历方式设计求解树高度问题的算法, 算法空间复杂度大  $O$  表示为  $O(\text{树的高度})$ 。( )
20. 树可以等价转化二叉树, 树的先序遍历序列与其相应的二叉树的前序遍历序列相同。( )
21. 如果一个连通无向图  $G$  中所有边的权值均不同, 则  $G$  具有唯一的最小生成树。( )
22. 求解最小生成树问题时, Kruskal 算法更适用于稀疏图, Prim 算法更适用于稠密图。( )
23. 使用线性探测法处理散列表碰撞问题, 若表中仍有空槽 (空单元), 插入操作一定成功。( )
24. 从链表中删除某个指定值的结点, 其时间复杂度是  $O(1)$ 。( )
25. 弗洛伊德 (Floyd) 算法是一种求解有向和无向图最短路径的动态规划算法, 但该算法的局限性是无法正确求解带有负权值边的图的最短路径。( )

## 三、填空题 (20 分, 每题 2 分)

26. 定义二叉树中一个结点的度数为其子结点的个数。现有一棵结点总数为 101 的二叉树, 其中度数为 1 的结点有 30 个, 则度数为 0 结点有 \_\_\_\_\_ 个。
27. 定义完全二叉树的根结点所在层为第一层, 如果一个二叉树的第六层有 23 个叶结点, 则它的总结点数量可能有 \_\_\_\_\_ 个。(请填写所有 3 个可能的结点数。)
28. 对于初始排序码序列 (51, 41, 31, 21, 61, 71, 81, 11, 91), 第 1 趟快速排序 (以第一个数字为 pivot 轴值) 的结果是: \_\_\_\_\_。
29. 如果输入序列是已经正序, 在 (改进) 冒泡排序、直接插入排序和直接选择排序算法中, \_\_\_\_\_ 算法最慢结束。
30. 已知某二叉树的先根周游序列为  $\{A, B, D, E, C, F, G\}$ , 中根周游序列为  $\{D, B, E, A, C, G, F\}$ , 则该二叉树的后根次序周游序列为  $\{\text{_____}\}$ 。
31. 使用栈计算后缀表达式 (操作数均为一位数)  $1\ 2\ 3\ +\ 4\ * \ 5\ +\ 3\ +\ -$ , 当扫描到第二个  $+$  号但还未对该  $+$  号进行运算时, 栈的内容 (以栈底到栈顶从左往右的顺序书写) 为 \_\_\_\_\_。
32. 51 个顶点的连通图  $G$  有 50 条边, 其中权值为 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 的边各 5 条, 则连通图  $G$  的最小生成树各边的权值之和为 \_\_\_\_\_。
33. 包含  $n$  个顶点无向图的邻接表存储结构中, 所有顶点的边表中最多有 \_\_\_\_\_ 个结点。具有  $n$  个顶点的有向图, 一个顶点入度和出度之和最大值不超过 \_\_\_\_\_。
34. 给定一个长度为 7 的空散列表, 采用双散列法解决冲突, 两个散列函数分别为  $h_1(key) = key \% 7$ ,  $h_2(key) = key \% 5 + 1$ 。请向散列表依次插入关键字为 30, 58, 65 的集合元素, 插入完成后 65 在散列表中存储地址为 \_\_\_\_\_。
35. 阅读 Python 算法 ABC, 回答问题。

```
from math import isqrt
```

```
def ABC(n):
    m = isqrt(n)
    for k in range(2, m + 1):
        if n % k == 0:
            return 0
    return 1
```

- (1) 算法的功能是：\_\_\_\_\_。
- (2) 算法的时间复杂度是  $O(\text{_____})$ 。

#### 四、简答题 (3 题, 共 14 分)

36. (字符串匹配与 KMP) 字符串匹配算法从长度为  $n$  的文本串  $S$  中查找长度为  $m$  的模式串  $P$  的首次出现位置。

- (1) 字符串匹配的朴素算法使用暴力搜索, 大致过程如下: 对于  $P$  在  $S$  中可能出现的  $n - m + 1$  个位置, 比对此位置时  $P$  和  $S$  中对应子串是否相等。其时间复杂度  $O((n - m + 1)m)$ 。请举例说明算法时间复杂度一种最坏情况 (注: 例子中请只出现 0 和 1 两种字符)。
- (2) 已知字符串  $S$  为 abaabaabacacaabaabcc, 模式串  $t$  为 abaabc。采用 KMP 算法进行查找, 请写出 next 数组。
- (3) 说明在上述 (2) 步骤中, 第一次出现不匹配 ( $s[i] \neq t[j]$ ) 时,  $i = j = 5$ ; 则下次比对时,  $i$  和  $j$  的值分别是?

37. (AOV 网络与拓扑排序) 有八项活动, 每项活动标记为  $V_n (0 \leq n \leq 7)$ , 每项活动要求的前驱如下:

| 活动 | V0  | V1 | V2 | V3     | V4 | V5     | V6 | V7     |
|----|-----|----|----|--------|----|--------|----|--------|
| 前驱 | 无前驱 | V0 | V0 | V0, V2 | V1 | V2, V4 | V3 | V5, V6 |

试画出相应的 AOV 网络, 并给出一个拓扑排序序列, 如存在多种, 则按照编号从小到大排序, 输出最小的一种。

38. (Huffman 树与 Huffman 编码) 简要回答下列 Huffman 树以及 Huffman 编码的相关问题。

- (1) 假定需要编码的字符个数为  $m$ , 哈夫曼树构造后结点总数多少? 简要阐述理由。
- (2) 假定报文中各字符及其出现次数为:  $A(50), B(30), C(10), D(25), E(11), F(99), G(8), H(51), I(22)$ 。试画出对应的 Huffman 树 (严格按照左小右大构造), 并给出各个字符的 Huffman 编码 (左分支编码 0, 右分支编码 1)。

#### 五、算法填空 (4 题, 共 26 分)

39. (有序链表删除重复元素) 请填空完成下列 Python 程序: 读入一个从小到大排好序的整数序列到链表, 然后在链表中删除重复的元素, 使得重复的元素只保留 1 个, 然后将整个链表内容输出。

输入: 第一行是正整数  $n (n < 80)$ , 第二行是  $n$  个整数。输入样例为 9 和 1 2 2 2 3 3 4 4 6, 输出样例为 1 2 3 4 6。

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

n = int(input())
a = list(map(int, input().split()))
head = Node(a[0])
p = head
for x in a[1:]:
    p.next = Node(_____) # (1)
    p = _____ # (2)
```

```

p = head
while p is not None:
    while _____ and p.data == p.next.data: # (3)
        _____ # (4)
    p = p.next

p = head
while p is not None:
    print(p.data, end=" ")
    _____ # (5)

```

40. (堆排序) 为实现对一组数据根据排序码进行自小到大排序, 请填空完成下列 Python 堆排序程序 (仅含关键部分代码)。每条记录用字典表示, 其中 `record["key"]` 为排序码。程序中建立的堆是大根堆 (即最大值堆, 最大元素在堆顶)。

```

def sift(records, i, n):
    temp = records[i]
    child = _____ # (1)
    while child < n:
        if _____ and records[child]["key"] < records[child + 1]["key"]: # (2)
            child += 1
        if temp["key"] < records[child]["key"]:
            _____ # (3)
            i = child
            child = _____ # (4)
        else:
            break
    records[i] = temp

def heap_sort(records):
    n = len(records)
    for i in range(n // 2 - 1, -1, -1):
        _____ # (5)
    for i in range(n - 1, 0, -1):
        records[0], records[i] = records[i], records[0]
        _____ # (6)

```

41. (AOV 网的拓扑排序程序) 以下 Python 代码实现了对 AOV 网的拓扑排序。设 `graph[i]` 存放从顶点  $i$  出发的所有邻接点编号, 请阅读代码并补全空缺。

```

def find_indegree(graph):
    indegree = [0] * len(graph)
    for i in range(len(graph)):
        for v in graph[i]:
            _____ # (1)
    return indegree

def topo_sort(graph):
    indegree = find_indegree(graph)
    stack = []
    topo = []
    for i in range(len(graph)):
        if indegree[i] == 0:
            _____ # (2)

```

```

while ____:                                # (3)
    j = stack.pop()
    topo.append(j)
    for k in ____:                          # (4)
        indegree[k] = ____                # (5)
        if indegree[k] == 0:
            ____                            # (6)

if ____:                                    # (7)
    print("The aov network has a cycle")
    return None
return topo

```

42. (无向图连通性与回路判断) 请填空完成下列 Python 程序：给定一个无向图，判断是否连通，是否有回路。图用邻接矩阵  $G$  表示， $G[v][u] == 1$  表示顶点  $v$  与顶点  $u$  相邻。

```

def dfs_connection(G, visited, v):
    visited[v] = True
    total = 1
    for u in range(len(G)):
        if G[v][u] and not visited[u]:
            total += dfs_connection(G, visited, u)
    return total

def dfs_loop(G, visited, v, parent):
    visited[v] = True
    for u in range(len(G)):
        if G[v][u]:
            if not visited[u]:
                if ____:                    # (1)
                    return True
            else:
                if ____:                    # (2)
                    return True
    return False

n, m = map(int, input().split())
G = [[0] * n for _ in range(n)]
for _ in range(m):
    a, b = map(int, input().split())
    G[a][b] = G[b][a] = 1

visited = [False] * n
total = dfs_connection(G, visited, 0)
if ____:                                  # (3)
    print("connected:yes")
else:
    print("connected:no")

visited = [False] * n
loop_found = False
for i in range(n):
    if ____:                              # (4)
        if dfs_loop(G, visited, i, -1):
            loop_found = True
            break

if loop_found:

```



```
    print("loop:yes")
else:
    print("loop:no")
```

# 数据结构与算法 B

## common-24 春-期末

### 一、选择题 (30 分, 每小题 2 分)

- 设栈的输入序列为 1, 2, 3, 4, 5, 下列序列中不可能是其出栈序列的是 ( )。
 

|           |           |
|-----------|-----------|
| (A) 23415 | (C) 23145 |
| (B) 54132 | (D) 15432 |
- 对线性表进行二分法查找, 其前提条件是 ( )。
 

|                                |                                |
|--------------------------------|--------------------------------|
| (A) 线性表以链接方式存储, 并且按关键码值排序      | (C) 线性表以顺序方式存储, 并且按关键码值排序      |
| (B) 线性表以链接方式存储, 并且按关键码值的检索频率排序 | (D) 线性表以顺序方式存储, 并且按关键码值的检索频率排序 |
- 对于二叉树中的结点  $N$ , 若其左子树高度为  $h_1$ , 右子树高度为  $h_2$ , 则以  $N$  为根结点的子树高度为 ( )。
 

|                      |                          |
|----------------------|--------------------------|
| (A) $\max(h_1, h_2)$ | (C) $\max(h_1, h_2) + 1$ |
| (B) $\min(h_1, h_2)$ | (D) $\min(h_1, h_2) + 1$ |
- 在插入排序算法中, 如果待排序序列已经是有序的, 则插入排序算法的时间复杂度为 ( )。
 

|                   |            |
|-------------------|------------|
| (A) $O(n^2)$      | (C) $O(n)$ |
| (B) $O(n \log n)$ | (D) $O(1)$ |
- 下列概念中属于存储结构的是 ( )。
 

|         |        |
|---------|--------|
| (A) 线性表 | (C) 栈  |
| (B) 链表  | (D) 队列 |
- 在二分查找算法中, 每次比较后都将搜索范围缩小一半。若要在长度为  $n$  的数组中查找一个元素, 最坏情况下需要比较多少次? ( )
 

|                           |                                    |
|---------------------------|------------------------------------|
| (A) $n$                   | (C) $\lfloor \log_2 n \rfloor$     |
| (B) $\lfloor n/2 \rfloor$ | (D) $\lfloor \log_2 n \rfloor + 1$ |
- 在一个具有  $n$  个结点的有序单链表中插入一个新结点并仍保持其有序, 平均时间复杂度为 ( )。
 

|                   |              |
|-------------------|--------------|
| (A) $O(n \log n)$ | (C) $O(n)$   |
| (B) $O(1)$        | (D) $O(n^2)$ |
- 设有 4 棵结点关键码均为整数的二叉树, 若其中序遍历序列分别如下, 可能是二叉搜索树的是 ( )。
 

|                   |                   |
|-------------------|-------------------|
| (A) 5, 3, 1, 2, 4 | (C) 5, 3, 4, 2, 1 |
| (B) 1, 2, 3, 4, 5 | (D) 1, 3, 5, 4, 2 |
- 线性表有  $2n$  ( $n > 100$ ) 个元素, 以下操作中, 哪一项在单链表上实现比在顺序表上实现效率更高? ( )

- (A) 在表中最后一个元素的后面插入一个新元素  
(B) 顺序输出表中的前  $i$  个元素
- (C) 交换表中第  $i$  个元素和第  $n + i$  个元素的值 ( $i = 0, \dots, n - 1$ )  
(D) 删除表中第 1 个元素
10. 用数组  $O[n]$  实现循环队列。设队头的前一个元素下标为  $f$ ，队尾下标为  $r$ ，则队列中元素总数为 ( )。
- (A)  $r - f$   
(B)  $(n + f - r) \bmod n$
- (C)  $n + r - f$   
(D)  $(n + r - f) \bmod n$
11. 一个拥有 21 条边的非连通无向图，至少包含 ( ) 个顶点。
- (A) 5  
(B) 6
- (C) 7  
(D) 8
12. 若使用分治算法解决某问题，每次把规模为  $n$  的问题分解为 2 个规模为  $n/2$  的子问题，并用  $O(n)$  时间合并两个子问题结果，则总时间复杂度为 ( )。
- (A)  $O(n)$   
(B)  $O(n \log n)$   
(C)  $O(n^2)$   
(D)  $O(n^2 \log n)$
13. 有  $n$  个顶点、 $m$  条边的无向图，下列说法中错误的是 ( )。
- (A) 采用邻接表存储，空间复杂度为  $O(n + m)$   
(B) 若为稠密图，更倾向于采用邻接矩阵存储  
(C) 采用邻接表存储，删除一个顶点的最坏情况时
- 间复杂度为  $O(n + m)$   
(D) 若为稀疏图，深度优先周游的时间复杂度低于广度优先周游的时间复杂度
14. 利用栈将表达式  $3 * 2^{(4 + 2 * 2 - 6 * 3)} - 5$  (其中  $\wedge$  为乘幂) 转换为后缀表达式。扫描到 6 时，运算符栈为 ( )。
- (A)  $*^{(+ -}$   
(B)  $*^{\wedge}$   
(C)  $*^{(+$   
(D)  $*^{(-$
15. 定义一棵没有 1 度结点的二叉树为满二叉树。对于一棵包含  $k$  个结点的满二叉树，其中叶子结点的个数为 ( )。
- (A)  $\lfloor k/2 \rfloor$   
(B)  $\lfloor k/2 \rfloor - 1$
- (C)  $\lfloor k/2 \rfloor + 1$   
(D) 以上三个都有可能

## 二、判断题 (10 分，每小题 1 分；对填 Y，错填 N)

16. 分治法和动态规划法都适用于将问题分解为规模较小的子问题的思想。( )
17. 在链表中查找和删除一个元素的时间复杂度都是  $O(1)$ 。( )
18. 相比于顺序存储，完全二叉树更适合用链式表示存储。( )
19. 通常不能仅通过一棵二叉树的前序遍历结点序列和后序遍历结点序列确定这棵二叉树的完整结构。( )

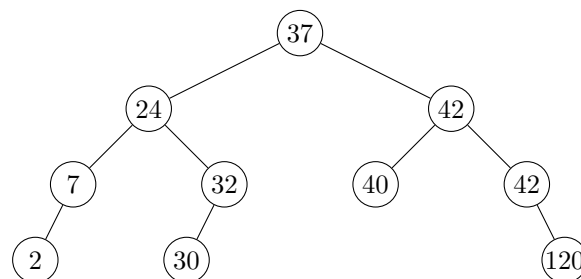
20. 二叉搜索树一定是满二叉树。( )
21. 归并排序算法一定比简单插入排序算法的执行效率高。( )
22. 在待排序元素为正序情况下，直接插入排序可能比快速排序的时间复杂度更小。( )
23. 一个有  $n$  个结点的无向图，最少有一个连通分量。( )
24. 图的遍历算法 (如深度优先搜索 DFS 和广度优先搜索 BFS) 都只能用于无向图。( )
25. 若连通无向图的边权值互不相同，其最小生成树只有一个。( )

### 三、填空题 (20 分，每题 2 分)

26. 若经常需要在线性表头部进行插入和删除操作，最合适的存储结构是 \_\_\_\_\_；若需要随机存取，最合适的存储结构是 \_\_\_\_\_。
27. 顺序表中  $a, b, c, d, e, f$  的查找概率分别为 0.12, 0.25, 0.23, 0.20, 0.05, 0.15。为了使查找成功的平均比较次数最少，数据元素的存放顺序应为 \_\_\_\_\_。
28. 含 120 个结点的完全二叉树共有 \_\_\_\_\_层，有 \_\_\_\_\_个叶子结点。
29. 已知一棵树的中序遍历序列为 DBGEACF、后序遍历序列为 DGEBFCA，其前序遍历结果为 \_\_\_\_\_。
30. 字符  $a, b, c, d$  的出现次数分别为 1000, 2000, 6000, 1000，采用 Huffman 编码时，该电文总长度为 \_\_\_\_\_比特。
31. 对关键字序列 45, 80, 55, 40, 42, 85 从最后一个非叶子结点开始调整建最大堆，得到的初始最大堆为 \_\_\_\_\_。
32. 一棵  $m$  叉树中度数为  $i$  的结点数为  $N_i$ ，则终端结点数为 \_\_\_\_\_。
33. 散列表长  $m = 15$ ，散列函数  $H(\text{key}) = \text{key} \% 11$ ，已有地址  $\text{addr}(16) = 5$ 、 $\text{addr}(37) = 4$ 、 $\text{addr}(50) = 6$ 、 $\text{addr}(70) = 7$ 。若用线性探查处理冲突，关键字 38 的地址为 \_\_\_\_\_。
34. 森林  $F$  有 4 棵树，结点数分别为 10, 9, 11, 7。转成一棵二叉树后，其根结点右子树中有 \_\_\_\_\_个结点。
35. 对无向图，若边数  $m \geq n(n$  为结点数)，则该图一定 \_\_\_\_\_。

### 四、简答题 (3 题，共 14 分)

36. 对序列  $(H, E, B, L, G, A, F, J, I, C, D, K)$  按字母升序排序，求：
  - (1) 冒泡排序第一趟交换结束后的结果；
  - (2) 二路归并排序第一趟归并后的结果；
  - (3) 初始步长为 4 的 Shell 排序第一趟后的结果。
37. 在题图给出的二叉排序树中删除结点 37。要求先寻找该结点左子树上的最大者  $r$ ，并利用  $r$  辅助删除该结点。请画出删除后的二叉排序树结构。



38. Dr. Stranger 的电脑感染了一种病毒。该病毒会把文档中的每种字母固定替换为另一种字母，且互不相同，可以看作字母集合上的双射函数。已知字母集合  $X = \{a, b, c, d, e\}$ ，文档感染后内容为

$$D' = \{\text{cedbdac}, \text{cac}, \text{ecd}, \text{dca}, \text{aba}, \text{bac}\}.$$

为使题目可唯一作答，补充条件：病毒替换规则是字母表  $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$  的循环置换，即

$$f(a) = b, \quad f(b) = c, \quad f(c) = d, \quad f(d) = e, \quad f(e) = a.$$

求解密函数  $f^{-1}$ ，并还原原文档  $D$ 。

## 五、算法填空 (4 题, 共 26 分)

39. (只带尾引用的循环单链表) 程序描述只带尾引用的循环单链表操作：先把  $n$  个整数依次插入链表头部，然后删除链表尾部  $m$  个元素，再在链表前部插入  $k$  个整数。补全程序空白。

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.tail = None          # 只保存尾结点引用

    def push_front(self, data): # 在链表头部插入元素
        nd = Node(data)
        if self.tail is None:
            self.tail = nd
            nd.next = nd
        else:
            nd.next = _____ # (1)
            self.tail.next = nd

    def pop_back(self):          # 在链表尾部删除元素
        if self.tail is None:
            return
        if _____:          # (2)
            self.tail = None
            return

        ptr = self.tail.next
        while _____:        # (3)
            ptr = ptr.next
        ptr.next = self.tail.next
        _____              # (4)

    def to_list(self):
        if self.tail is None:
            return []
        ans = []
        ptr = self.tail.next
        while True:
            ans.append(ptr.data)
            ptr = ptr.next
            if _____:        # (5)
                break
        return ans
```

40. (二叉树宽度) 给定一棵二叉树，宽度定义为结点最多的那一层的结点数。输入每个结点的左右儿子编号，若为  $-1$  表示没有相应儿子，补全程序求二叉树宽度。

```
class BinaryTree:
    def __init__(self, data=None):
```

```

        self.data = data
        self.left = None
        self.right = None
        self.parent = -1

def traversal(root, level):
    if root is None:
        return
    width[level] += 1
    ----- # (1)
    ----- # (2)

n = int(input())
nodes = [BinaryTree(i) for i in range(n)]
width = [0] * max(1, n)
for nd in range(n):
    L, R = map(int, input().split())
    nodes[nd].left = None if L == -1 else nodes[L]
    nodes[nd].right = None if R == -1 else nodes[R]
    if L != -1:
        ----- # (3)
    if R != -1:
        ----- # (4)

root = None
for i in range(n):
    if -----: # (5)
        root = nodes[i]
        break

traversal(root, 0)
print(max(width))

```

41. (Prim 算法) 用 Prim 算法求无向图最小生成树。图用邻接矩阵  $G$  存储,  $G[u][v] = \text{None}$  表示无边, 顶点编号从 0 开始。补全程序, 使其返回最小生成树的边列表。

```

def prim(G):
    INF = float("inf")
    n = len(G)
    dist = [INF] * n # 各顶点到已建树部分的最短距离
    used = [False] * n # 顶点是否已加入最小生成树
    prev = [None] * n # 顶点 v 通过边 (v, prev[v]) 加入树
    edges = []
    done_num = 0

    while -----: # (1)
        if done_num == 0:
            x, min_dist = 0, 0
        else:
            x, min_dist = None, INF
            for i in range(n):
                if (not used[i]) and -----: # (2)
                    x, min_dist = i, dist[i]

            ----- # (3)
        done_num += 1
        if done_num > 1:

```

```

edges.append(_____) # (4)

for v in range(n):
    if (not used[v]) and G[x][v] is not None and _____: # (5)
        dist[v] = G[x][v]
        _____ # (6)

return edges

```

42. (连通分量大小) 补全程序，计算一个无向图中所有连通分量的结点数。输入为图的结点数和边列表，输出每个连通分量的结点数。

```

def dfs(node, visited, graph, n):
    count = 1
    visited[node] = True
    for i in range(n):
        if i in graph[node] and _____: # (1)
            count += _____ # (2)
    return count

def count_components(n, edges):
    graph = {i: [] for i in range(n)}
    for u, v in edges:
        _____ # (3)
        _____ # (4)

    visited = [False] * n
    components = []
    for i in range(n):
        if _____: # (5)
            components.append(_____) # (6)
    return components

n = int(input())
edges = eval(input())
print(count_components(n, edges))

```

# 数据结构与算法 B

## common-模拟题 1-期末

### 一、选择题 (每小题 3 分, 共 30 分)

1. 数据结构的三个基本要素是 ( )。

- (A) 顺序结构、链式结构、散列结构
- (B) 逻辑结构、存储结构、算法
- (C) 线性结构、树型结构、图状结构
- (D) 以上都不是

2. 栈是一种后进先出表。若入栈的顺序为  $\{1, 2, 3, 4\}$ , 则出栈的顺序不可能是 ( )。

- (A) 1234
- (B) 1243
- (C) 1423
- (D) 2134

3. 用一个大小为 6 的数组来实现环形队列, 元素从下标 0 开始存放。当前 front 和 rear 的值分别为 4 和 1, 现在从队列中删除一个元素, 再增加两个元素, 则 front 和 rear 的值为 ( )。

- (A) 0, 2
- (B) 5, 3
- (C) 0, 1
- (D) 5, 2

4. 假定一棵二叉树的结点个数为 33 个, 定义根结点的层数为 0, 则它的高度最小为 ( )。

- (A) 5
- (B) 6
- (C) 7
- (D) 4

5. 已知某二叉树的先根周游序列为  $\{A, B, D, E, C\}$ , 中根周游序列为  $\{D, B, E, A, C\}$ , 则该二叉树的后根周游序列是 ( )。

- (A)  $\{A, B, C, D, E\}$
- (B)  $\{D, B, E, C, A\}$
- (C)  $\{C, D, E, B, A\}$
- (D)  $\{D, E, B, C, A\}$

6. 在待排序的元素序列基本有序的前提下, 下列空间效率最差的排序方法是 ( )。

- (A) 直接插入排序
- (B) 选择排序
- (C) 快速排序
- (D) 冒泡排序

7. 在平均情况下, 下列时间复杂度最好的排序方法是 ( )。

- (A) 直接插入排序 (平均  $O(n^2)$ )
- (B) 选择排序 (平均  $O(n^2)$ )
- (C) 快速排序 (平均  $O(n \log n)$ )
- (D) 冒泡排序 (平均  $O(n^2)$ )

8. 下列关于图的说法不正确的是 ( )。

- (A) 用邻接矩阵法存储一个图时, 其存储空间与图的边数无关
- (B) AOV 网络拓扑排序的结果是唯一的
- (C) 强连通分量是有向图中的极大强连通子图
- (D) 一个有向图的邻接表和逆邻接表中的结点个数一定相等



9. AVL 树是一种 ( )。

- (A) 生成树
- (B) 平衡树

- (C) B 树
- (D) 二叉排序树

10. 任何一个无向连通图的最小生成树 ( )。

- (A) 只有一棵
- (B) 有一棵或多棵

- (C) 一定有多棵
- (D) 可能不存在

二、解答题 (共 40 分)

11. (线性表存储结构比较) (8 分)

请说明线性表的顺序存储 (顺序表) 与链式存储 (单链表) 的异同, 比较它们的优缺点。当字典采用二分法进行检索时, 应采用哪一种存储结构?

12. (Huffman 编码) (10 分)

字符串 "this sentence is used for test", 请使用 Huffman 编码方式对该字符串中的所有字符重新编码, 以实现对它的压缩, 给出编码过程中实现的 Huffman 树以及字符 's' 的编码结果。

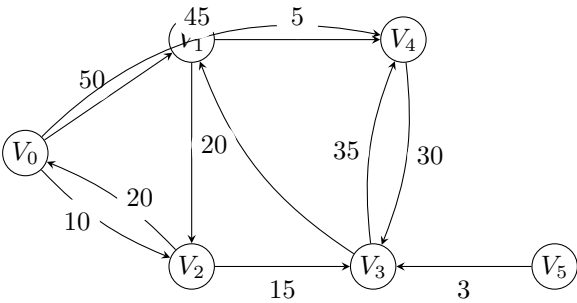
13. (排序算法) (12 分)

对待排序的关键码集合  $\{E, C, A, M, S, R, D, F\}$  按升序排序, 请叙述直接插入排序、堆排序、快速排序、归并排序的基本思路, 以及它们对上述数组第一趟扫描后的结果 (快速排序使用第一个元素作为分界点)。

14. (最短路径) (10 分)

下面是一个带权有向图的邻接矩阵表示 ( $\infty$  表示无边), 试画出该图结构, 并给出该图中由  $V_0$  到  $V_4$  的最短路径。

$$\begin{pmatrix} 0 & 50 & 10 & \infty & 45 & \infty \\ \infty & 0 & 15 & \infty & 5 & \infty \\ 20 & \infty & 0 & 15 & \infty & \infty \\ \infty & 20 & \infty & 0 & 35 & \infty \\ \infty & \infty & \infty & 30 & 0 & \infty \\ \infty & \infty & \infty & 3 & \infty & 0 \end{pmatrix}$$



### 三、算法填空 (共 10 分)

#### 15. (散列表检索) (10 分)

完成下列 Python 散列表检索算法。设散列函数为 `h`，采用线性探查法解决冲突；`EMPTY` 表示空位置。函数返回二元组 (`found`, `position`)，其中 `found` 表示是否检索成功，`position` 表示查找成功位置或首次空位位置。

```
EMPTY = None

def linear_search(table, key, h):
    m = len(table)
    d = _____ # (1) 初始散列地址
    for _ in range(m):
        entry = table[d]
        if entry is not EMPTY and _____: # (2) 检索成功
            _____ # (3) 记录位置
            return True, position
        if entry is EMPTY:
            return False, d
        _____ # (4) 线性探查下一地址
    _____ # (5) 散列表已探查一周
```

## 四、算法设计 (共 20 分)

## 16. (求子树深度) (20 分)

已知一棵树采用孩子兄弟表示法。请用 Python 设计函数，求树中以值为  $x$  的结点为根的子树深度。限定：值为  $x$  的结点在树中最多有一个。注意栈或队列等辅助结构可以直接引用，不用自己定义。

```
class CSNode:
    def __init__(self, info):
        self.info = info          # 结点数据
        self.first_child = None   # 第一个孩子
        self.next_sibling = None  # 下一个兄弟

p_mytree = ...                  # 树根引用
```